

Projet : Implémentation et expérimentation d'un schéma de type splitting pour l'équation de Schrödinger non linéaire stochastique

Eowyn Renoud Hallereau, 5ème année MAM, Polytech Lyon

11 février 2026

Table des matières

Introduction	3
1 Équation de Schrödinger	4
1.1 Équation déterministe	4
1.2 Équation non linéaire	4
1.3 Équation stochastique	4
1.4 Différents schémas possibles	5
2 Résolution numérique de l'équation	6
2.1 Schéma de type splitting	6
2.2 Méthode pseudo-spectrale	6
2.3 Points importants	7
3 Résultats numériques	8
3.1 Euler-Maruyama	8
3.2 Potentiel externe	9
3.3 Interaction non locale cubique	12
Conclusion	14
A Programmes	15
A.1 Fonctions générales	15
A.2 Euler-Maruyama	16
A.3 Potentiel externe	17
A.4 Interaction non locale	20
Bibliographie	22

Table des figures

3.1	Solution par le schéma d'Euler-Maruyama (potentiel externe)	9
3.2	Espérance de la masse obtenue par la formule théorique et le schéma d'Euler-Maruyama	9
3.3	Solution par le schéma de splitting (potentiel externe)	9
3.4	$ u $ au cours du temps (schéma de splitting, potentiel externe)	10
3.5	Espérance de la masse obtenue par la formule théorique et le schéma de splitting . . .	10
3.6	$ u $ schéma de splitting et déterministe (potentiel externe)	11
3.7	Solution par différents schémas (potentiel externe)	11
3.8	Espérance de la masse obtenus par différents schémas	12
3.9	$ u $ schéma de splitting (interaction non locale)	12
3.10	Solution par différents schémas (interaction non locale)	13
3.11	Espérance de la masse obtenue par différents schémas	13

Introduction

Dans le cadre de ma dernière et 5^{ème} année à Polytech, en spécialité Mathématiques Appliquées et Modélisation, j'ai travaillé sur le projet "Implémentation et expérimentation d'un schéma de type splitting pour l'équation de Schrödinger non linéaire stochastique". Mon tuteur était Monsieur Honoré.

J'ai choisi de m'intéresser à ce sujet car l'équation de Schrödinger est une équation différentielle partielle avec laquelle je n'avais encore jamais travaillé. De plus c'est une équation avec de nombreuses applications, et avec des membres non linéaires et des perturbations stochastiques, la rendant plus complexe. Cette équation est notamment utilisée en mécanique quantique, optique non linéaire et en physique des milieux dispersifs.

Cette équation non linéaire et stochastique est très rarement solvable analytiquement, il faut donc mettre en œuvre des méthodes numériques. Plusieurs méthodes existent mais une des plus performantes, spécialement pour l'équation de Schrödinger non linéaire stochastique, est un schéma de type splitting. En effet ces schémas permettent d'obtenir des ordres de convergence élevés et de conserver des propriétés de la solution analytique, détaillées plus tard.

L'objectif de ce projet est donc d'implémenter un schéma de splitting pour l'équation de Schrödinger non linéaire stochastique, avec différentes non linéarités. La discrétisation spatiale sera effectuée par une méthode pseudo-spectrale, mettant en œuvre la transformée de Fourier. L'objectif est aussi de comparer les résultats obtenus à ceux obtenus par l'article donné en référence [1]. De plus, on comparera les résultats du schéma de splitting à d'autres schémas temporels.

La première partie de ce rapport présente l'équation de Schrödinger sous ses différentes formes, et les différents schémas numériques envisageables. La deuxième partie présente les méthodes mises en place pour la résolution numérique de l'équation. La dernière partie analyse les résultats numériques obtenus.

Remerciements

Je remercie sincèrement mon tuteur Monsieur Honoré pour son accompagnement et son aide tout au long de ce projet. Je remercie également Madame Débit pour ses commentaires lors de la présentation à mi-parcours.

Chapitre 1

Équation de Schrödinger

1.1 Équation déterministe

L'équation de Schrödinger est une équation fondamentale de la physique quantique. Elle a la même importance que la seconde loi de Newton pour la mécanique classique. C'est une équation qui permet notamment de décrire la propagation des ondes. L'exemple le plus simple et le plus courant est le cas d'une particule dans une boîte. Mais cette équation a des applications dans de nombreux domaines, comme la physique des plasmas, l'optique et en électrostatique [2]. On s'intéressera ici à l'équation non linéaire, qui correspond mieux à la réalité.

1.2 Equation non linéaire

L'équation de Schrödinger non linéaire peut s'écrire :

$$i\frac{\partial u}{\partial t}(x, t) = \Delta u(x, t) + F(x, u). \quad (1.1)$$

Il y a plusieurs non-linéarités souvent utilisées [3], elles sont principalement choisies pour leur sens mathématiques. On s'intéressera plus particulièrement au cas d'un potentiel externe, où F est définie par :

$$F(x, u) = V(x)u. \quad (1.2)$$

On s'intéressera également au cas correspondant à une interaction non locale cubique :

$$F(x, u) = (V(x) \star |u|^2)u. \quad (1.3)$$

Par exemple l'équation de Schrödinger non linéaire peut permettre de simuler la propagation de la lumière en optique non linéaire.

1.3 Equation stochastique

De plus, nous allons ici implémenter l'équation de Schrödinger non linéaire stochastique, qui permet de décrire la propagation d'ondes non linéaires dispersives, dans des milieux non-homogènes ou aléatoires [4, 5, 6]. L'inconnue $u(x, t)$ représente alors un champ ondulatoire. Le bruit sera ici représenté par une force motrice aléatoire, correspondant à un bruit additif. On obtient donc l'équation différentielle partielle stochastique :

$$i\frac{\partial u}{\partial t}(x, t) = \Delta u(x, t) + F(x, u) + \xi(x, t), \quad (1.4)$$

avec l'inconnue $u(t)$ un processus stochastique, et $x \in \mathbb{T}^d$ le tore de dimension d .

Plus précisément l'équation considérée sera :

$$\begin{cases} i du(t) &= \Delta u(t)dt + F(u(t))dt + \alpha dW^Q(t), \\ u(0) &= u_0. \end{cases} \quad (1.5)$$

La non linéarité $F = V[u]u$ peut prendre les différentes formes mentionnées ci-dessus (1.2) et (1.3). Le paramètre $\alpha > 0$ correspond à la taille du bruit. Le bruit est un processus de Wiener défini par :

$$W^Q(t) = \sum_{k \in \mathbb{N}} \gamma_k \beta_k(t) e_k, \quad (1.6)$$

où $(e_k)_{k \in \mathbb{N}} = \left(\frac{1}{\sqrt{2\pi}} e^{ikx} \right)_{k \in \mathbb{N}}$ est une base orthonormée de L^2 , $(\beta_k)_{k \in \mathbb{N}}$ est un mouvement Brownien standard et $(\gamma_k)_{k \in \mathbb{N}} = \left(\frac{1}{1+k^2} \right)_{k \in \mathbb{N}}$ sert à restreindre le bruit.

La solution de (1.5) sous forme intégrale est donnée par :

$$u(t) = S(t)u_0 - i \int_0^t S(t-s)F(u(s)) ds - i\alpha \int_0^t S(t-s) dW^Q(s), \quad (1.7)$$

où $S(t) = e^{-it\Delta}$.

Voir par exemple [7] pour une comparaison des bruits additifs et multiplicatifs. L'existence des solutions pour l'équation de Schrödinger stochastique non linéaire a été étudiée précédemment, par exemple dans [5].

1.4 Différents schémas possibles

Plusieurs schémas de discrétisation temporelle ont été testés et étudiés pour résoudre cette équation. Par exemple un des schémas le plus simple est le schéma d'Euler-Maruyama :

$$u_{n+1} = u_n - i\tau \Delta u_n - i\tau F(u_n) - i\alpha \delta W_n^Q, \quad (1.8)$$

avec $\delta W_n^Q = W^Q((n+1)\tau) - W^Q(n\tau)$ des incréments de Wiener. Des résultats obtenus avec le schéma de Crank-Nicholson ont également été souvent étudiés, voir par exemple [8]. Un schéma exponentiel stochastique [9, 10] peut également être utilisé :

$$u_{n+1} = S(\tau) \left(u_n - i\tau F(u_n) - i\alpha \delta W_n^Q \right). \quad (1.9)$$

Les schémas de type splitting sont également beaucoup utilisés pour les équations déterministe de Schrödinger, et sont très efficaces. Ils ont aussi été utilisés pour des équations stochastiques de Schrödinger [11]. Notamment, il existe déjà des résultats de convergence pour un schéma de splitting appliqué à une équation de Schrödinger non linéaire stochastique, dans le cas d'une interaction non locale cubique et d'une dispersion de bruit blanc [12]. Cependant, le but de [1] est de faire une analyse numérique d'un schéma de splitting approchant une solution sous forme intégrale de l'équation de Schrödinger non linéaire stochastique avec un bruit additif.

Chapitre 2

Résolution numérique de l'équation

2.1 Schéma de type splitting

Un schéma de type splitting est un schéma permettant une discrétisation temporelle. Il consiste à décomposer l'équation en sous-équations qui peuvent être résolues de manière exacte, ou plus efficacement numériquement.

Premièrement, on cherche à résoudre la partie non linéaire de l'équation (1.5) : $i \frac{\partial u(t)}{\partial t} = F(u(t)) = V[u]u$. Pour cela définissons le flux associé à cette équation différentielle, pour tout $u \in L^2$ et $t \in \mathbb{R}$:

$$\Phi_t(u) = e^{-itV[u]}u. \quad (2.1)$$

On peut voir que $i \frac{\partial \Phi_t(u)}{\partial t} = V[u]\Phi_t(u) = F(\Phi_t(u))$.

Puis la récursion explicite suivante est définie :

$$u_{n+1} = S(\tau) \left(\Phi_\tau(u_n) - i\alpha\delta W_n^Q \right), \quad (2.2)$$

avec $\tau \in [0, 1]$ le pas de temps. Ce schéma vient de l'application d'un schéma d'Euler exponentiel à l'équation $idu = \Delta u dt + \alpha\delta W^Q(t)$.

Ainsi le schéma de splitting consiste à faire les deux étapes suivantes pour chaque pas de temps. Premièrement, poser $\widetilde{u}_n = \Phi_\tau(u_n)$, ce qui consiste à résoudre exactement l'équation $i \frac{\partial u}{\partial t} = F(u)$ avec la condition initiale u_n . Deuxièmement, poser $u_{n+1} = S(\tau) \left(\widetilde{u}_n - i\alpha\delta W_n^Q \right)$.

Maintenant, détaillons une des raisons pour laquelle le schéma de splitting est intéressant. Pour des conditions périodiques, la masse est définie par :

$$M(u) = \|u\|_{L^2}^2 = \int |u|^2 dx, \quad (2.3)$$

c'est une quantité conservé pour l'équation de Schrödinger : $i \frac{\partial u}{\partial t} - \Delta u - V[u]u = 0$. Dans le cas stochastique, la solution exacte de l'équation de Schrödinger non linéaire stochastique, avec des non linéarités suffisamment régulières satisfait la formule de la trace suivante :

$$\mathbb{E}(M(u(t))) = \mathbb{E}(M(u_0)) + t\alpha^2 \text{Tr}(Q) \quad \forall t, \quad (2.4)$$

avec pour condition $\mathbb{E}(M(u_0)) < \infty$.

Le schéma de splitting a la particularité que la solution qu'il donne satisfait cette même formule, ainsi :

$$\mathbb{E}(M(u_n)) = \mathbb{E}(M(u_0)) + t_n\alpha^2 \text{Tr}(Q) \quad \forall t_n = n\tau. \quad (2.5)$$

2.2 Méthode pseudo-spectrale

La discrétisation spatiale est effectuée par une méthode pseudo-spectrale. Plus précisément la transformée de Fourier est utilisée pour résoudre la deuxième partie de l'équation. La transformée de Fourier est définie par :

$$\mathcal{F}(x) = \int_{\mathbb{R}} f(t) e^{-itx} dt. \quad (2.6)$$

D'après les propriétés de la transformée et la définition de $S(t)$, on a :

$$\mathcal{F}(S(t)u)(x) = S(t)\mathcal{F}(u)(x). \quad (2.7)$$

Ainsi, la deuxième étape du schéma de splitting peut s'écrire :

$$u_{n+1} = \mathcal{F}^{-1} \left(S(\tau) \mathcal{F}(\widetilde{u}_n - i\alpha\delta W_n^Q) \right), \quad (2.8)$$

avec $\widetilde{u}_n = \Phi_\tau(u_n) = e^{-i\tau V[u]}u$.

2.3 Points importants

Plusieurs points sont à détailler concernant la résolution numérique de l'équation de Schrödinger non linéaire stochastique avec un schéma de type splitting.

Le premier point est la simulation du processus de Wiener. Dans notre cas W^Q (1.6) est composé d'un mouvement brownien standard $(\beta_k)_{k \in \mathbb{N}}$, d'un facteur $(\gamma_k)_{k \in \mathbb{N}}$ permettant de restreindre le brownien, et $(e_k)_{k \in \mathbb{N}}$ des vecteurs d'une base orthonormée de L^2 . Les vecteurs e_k sont nécessaires pour la discrétisation spatiale du mouvement de Wiener sur tout l'espace, ici le tore. Rappelons que $(\beta_k)_{k \in \mathbb{N}}$ est un processus stochastique qui vérifie :

- i) $\mathbb{P}(\beta_0 = 0) = 1$,
- ii) $\forall n \in \mathbb{N}, \forall 0 < t_0 < \dots < t_n$ les variables $(\beta_{t_i} - \beta_{t_{i-1}})_{1 \leq i \leq n}$ sont indépendantes,
- iii) $\forall 0 < s < t, \beta_t - \beta_s \sim \mathcal{N}(0, t - s)$.

Il faut donc simuler ce processus selon une discrétisation spatiale en N_x points, et une discrétisation temporelle en N_t points, avec un pas τ . Premièrement, on génère les valeurs de γ_k où $k \in [0, Nx]$, puis les vecteurs e_k . Le mouvement brownien standard est simulé grâce à la loi normale $\mathcal{N}(0, \tau \mathbf{I}_{N_t \times N_x})$. Il faut ensuite préciser $\beta_0 = 0 \forall x$.

Un autre point important à préciser est l'implémentation de l'opérateur de convolution, notamment pour la non linéarité $F(u) = (V \star |u|^2)u$. Le choix a été fait d'utiliser la transformée de Fourier qui permet de transformer la convolution en un produit. En effet, si $f, g \in L^2$ alors $\mathcal{F}(f \star g) = \mathcal{F}(f)\mathcal{F}(g)$. Ainsi on a :

$$F(u) = (V \star |u|^2)u = \mathcal{F}^{-1}(\mathcal{F}(V)\mathcal{F}(|u|^2))u. \quad (2.9)$$

De la même façon, la transformée de Fourier est appliquée pour implémenter la convolution entre S et $\Phi_t(u)$. Ceci est détaillé plus haut dans la section 2.2. Il faut cependant faire attention, les transformées de Fourier sont discrétisées dans un domaine fréquentiel. Or la solution u est discrétisée dans le domaine spatial selon un pas Δx . Le pas du domaine fréquentiel est donné par $\Delta k = \frac{2\pi}{N_x \Delta x}$. Dans notre cas du tore :

$$\Delta x = \frac{2\pi}{N_x} \quad \text{et} \quad \Delta k = \frac{2\pi}{N_x \Delta x} = 1. \quad (2.10)$$

Enfin, pour le schéma discrétisé 2.8, on obtient $S(\tau) = e^{-i\tau k^2}$, $k \in \llbracket \frac{-N_x}{2}; \frac{N_x}{2} \rrbracket$.

Chapitre 3

Résultats numériques

Les simulations suivantes ont été réalisées avec le langage Python et les bibliothèques Numpy et Matplotlib. Le but de cette partie est de détailler les résultats obtenus et de les comparer avec les résultats obtenus dans l'article de référence [1], donc les paramètres seront choisis similaires. On travaillera exclusivement en une dimension. Les paramètres par défaut, si le contraire, n'est pas précisé sont : $N_x = 2^8$, $x \in [0, 2\pi]$ périodique avec $\Delta x = \frac{2\pi}{N_x} \simeq 0.0245$, $\tau = 0.1$, $t \in [0, T]$ avec $T = 1$ et $N_t = 11$. De plus le paramètre devant le bruit sera $\alpha = 1$. Les espérances seront calculées avec un algorithme de Monte Carlo à 500 itérations. C'est-à-dire qu'on utilisera $\frac{1}{500} \sum_{k=0}^{500} M_k(u)$.

3.1 Euler-Maruyama

En premier le schéma d'Euler-Maruyama (1.8) est implémenté. C'est un schéma d'Euler étendu pour correspondre aux équations différentielles stochastiques. On s'intéressera ici seulement à la non linéarité d'un potentiel externe (1.2) avec

$$V(x) = \frac{3}{5 - 4 \cos(x)}, \quad (3.1)$$

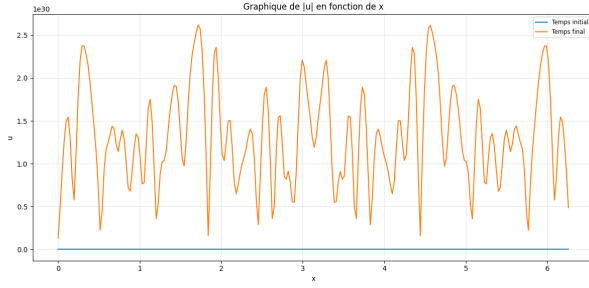
et la condition initiale déterministe :

$$u_0(x) = \frac{2}{2 - \cos(x)}. \quad (3.2)$$

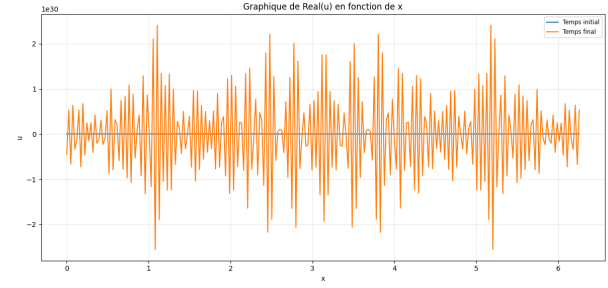
Pour pouvoir implémenter le schéma, il faut définir la matrice du laplacien discret en une dimension de taille $N_x \times N_x$:

$$\begin{bmatrix} 2 & -1 & & (0) \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ (0) & & -1 & 2 \end{bmatrix}. \quad (3.3)$$

La figure 3.1a affiche le module de la solution obtenue par le schéma. La figure 3.1b affiche la partie réelle de la solution. Les deux permettent d'illustrer la divergence de ce schéma pour l'équation de Schrödinger non linéaire stochastique. Ce schéma est le plus simple possible pour résoudre une EDS, il a été utilisé pour résoudre l'équation de Schrödinger stochastique contrôlé dans [13]. Mais pour notre équation non linéaire il n'est pas suffisant.



(a) $|u|$



(b) $Re(u)$

FIGURE 3.1 – Solution par le schéma d'Euler-Maruyama (potentiel externe)

De plus, le schéma d'Euler-Maruyama est un schéma qui ne respecte pas le comportement à long terme de la masse, donné par la formule de la trace (2.5). La figure 3.2 montre en effet une explosion exponentielle de l'espérance de la masse. Ceci est parfaitement cohérent avec les résultats de [1].

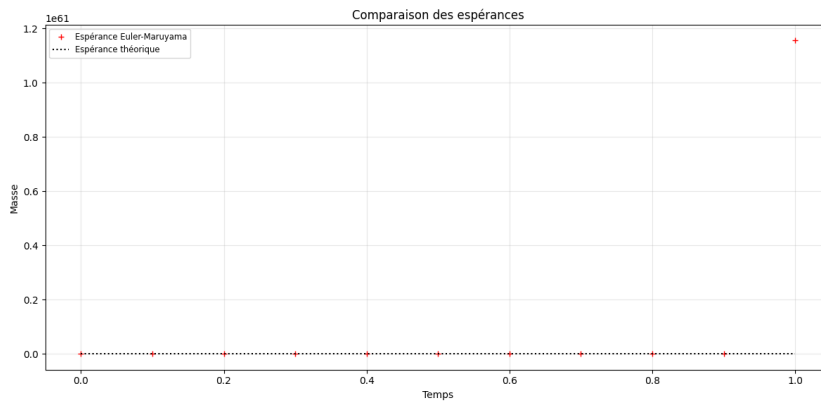


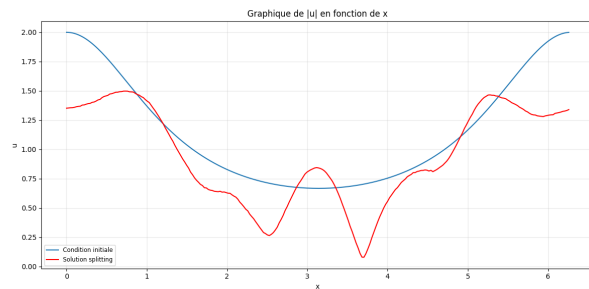
FIGURE 3.2 – Espérance de la masse obtenue par la formule théorique et le schéma d'Euler-Maruyama

Ce schéma ne sera donc plus utilisé pour les prochaines simulations.

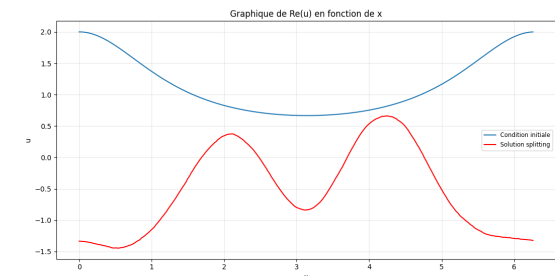
3.2 Potentiel externe

Etudions maintenant les résultats obtenus avec le schéma de splitting 2.2 associé à la méthode pseudo-spectrale. Premièrement, la non linéarité est donnée par un potentiel externe (1.2) avec $V(x) = \frac{3}{5-4\cos(x)}$ et la condition initiale est : $u_0(x) = \frac{2}{2-\cos(x)}$.

La figure 3.3a montre le module de la solution obtenue par le schéma de splitting, et la figure 3.3b sa partie réelle. La figure 3.4 illustre l'évolution de la solution au cours du temps. On peut observer que le schéma de splitting permet d'obtenir des résultats cohérents contrairement au schéma d'Euler-Maruyama.



(a) $|u|$



(b) $Re(u)$

FIGURE 3.3 – Solution par le schéma de splitting (potentiel externe)

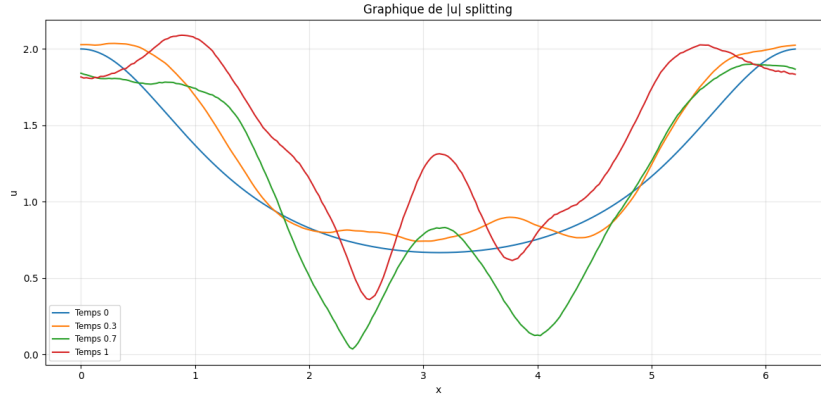


FIGURE 3.4 – $|u|$ au cours du temps (schéma de splitting, potentiel externe)

Étudions maintenant l'évolution de la masse pour le schéma de splitting. Pour ce résultat, 7500 itérations de Monte Carlo ont été effectuées, afin d'obtenir un résultat plus précis et de correspondre à l'article [1]. On trace à la fois l'espérance obtenue par la formule théorique (2.5) ainsi que celle calculée à chaque itération. On peut observer sur la figure 3.5 que la masse du schéma de splitting satisfait presque parfaitement la formule théorique (2.5) comme prévue. En calculant la norme infinie entre l'espérance de la masse par le schéma de splitting et celle théorique, on obtient :

$$\|E(M(u_{n_{sp}})) - E(M(u_{n_{th}}))\|_{\infty} = 6.62e^{-2}. \quad (3.4)$$

Les courbes obtenues ont donc le même comportement que dans [1], cependant l'ordre de grandeur est différent. Dans l'article, l'espérance de la masse varie entre 0 et $4e^{-3}$, alors qu'ici (figure 3.5) elle varie entre 9.6 et 11. Cela peut s'expliquer en supposant que les auteurs de [1] aient normalisée la masse, ou qu'un facteur d'échelle ait été appliqué.

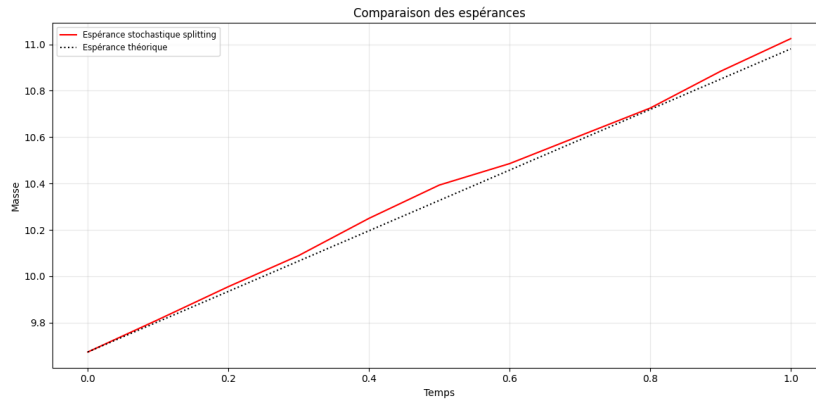
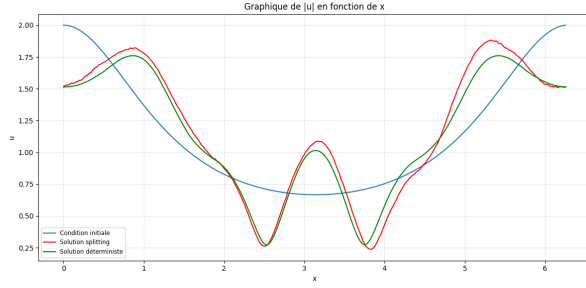


FIGURE 3.5 – Espérance de la masse obtenue par la formule théorique et le schéma de splitting

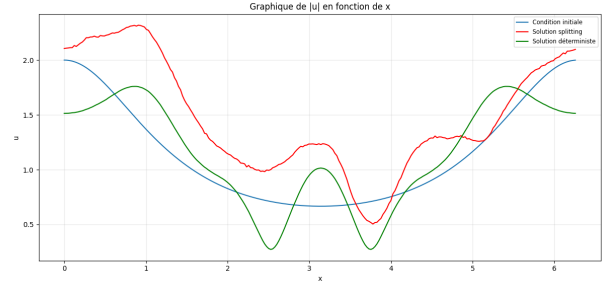
On va maintenant implémenter le même schéma de splitting, mais pour l'équation déterministe :

$$i \frac{\partial u}{\partial t} = \Delta u + F(u). \quad (3.5)$$

On veut ainsi observer les effets du terme stochastique sur la solution. Pour cela l'équation de Schrödinger non linéaire stochastique est résolue pour $\alpha = 1$ et pour $\alpha = 2$. La figure 3.6 permet d'observer les variations dues au terme stochastique. On peut notamment noter l'augmentation de ces variations quand $\alpha = 2$.



(a) $\alpha = 1$



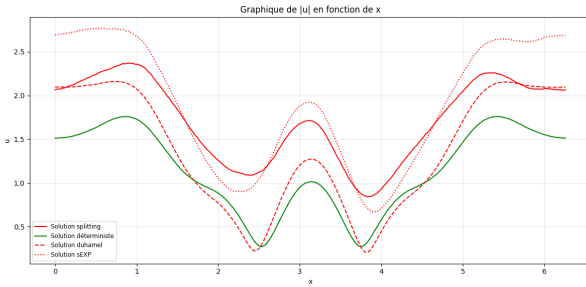
(b) $\alpha = 2$

FIGURE 3.6 – $|u|$ schéma de splitting et déterministe (potentiel externe)

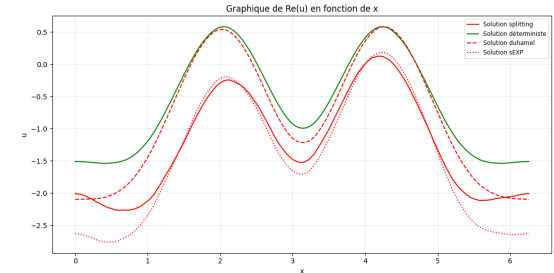
On souhaite maintenant comparer les résultats obtenus par le schéma de splitting avec ceux obtenus par la formule exacte(1.7), et par un schéma exponentiel stochastique donné par (1.9). La formule exacte discrétisé peut s'écrire :

$$u_n = S(\tau)u_0 - i \sum_{l=0}^n [S(n\tau - l\tau)F(u(l\tau))] \tau - i\alpha \sum_{l=0}^n [S(n\tau - l\tau)\delta W^Q(l)] \tau. \quad (3.6)$$

Les figures 3.7a et 3.7b illustrent les solutions au temps $T = 1$ pour les différentes méthodes. Il est possible de voir qu'elles convergent toutes, mais leurs résultats varient légèrement. On peut notamment remarquer que la solution par la formule exacte obtient un résultat proche de celle déterministe, même si elle n'est pas déterministe. Le schéma exponentiel stochastique quant à lui donne un résultat proche du schéma de splitting.



(a) $|u|$



(b) $Re(u)$

FIGURE 3.7 – Solution par différents schémas (potentiel externe)

Intéressons nous à l'évolution de la masse pour le schéma exponentiel stochastique. On observe sur la figure 3.8 une croissance plus rapide pour ce schéma. Ce résultat est cohérent avec [1] qui obtiennent également une sur-estimation de la croissance linéaire. On peut noter que l'espérance de la masse pour le schéma déterministe est constante, comme attendu.

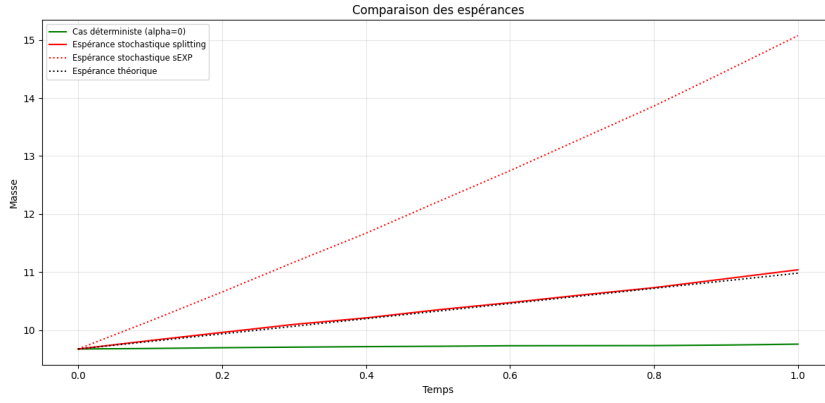


FIGURE 3.8 – Espérance de la masse obtenus par différents schémas

3.3 Interaction non locale cubique

Dans cette partie on s'intéresse à l'équation de Schrödinger non linéaire stochastique avec une non linéarité due à une interaction non locale cubique (1.3). On pose $V(x) = \cos(x)$ et $u_0 = \frac{1}{1+\sin(x)^2}$. La convolution de la non linéarité est traitée par des transformées de Fourier, comme expliqué dans la section 2.3. On applique le même schéma de splitting (2.2) associé à la méthode pseudo-spectrale que précédemment. Le but est de comparer les résultats avec l'article de référence [1], les résultats pour cette non linéarité ont également été comparés avec [7].

La figure 3.9 illustre le module de la solution obtenue par le schéma de splitting, pour $\alpha = 1$ et $\alpha = 2$. Par rapport au potentiel externe, on observe de plus grandes variations dues au terme stochastique. On peut aussi observer de petites oscillations qui peuvent être causées par le terme stochastique, mais aussi par les approximations des transformées de Fourier.

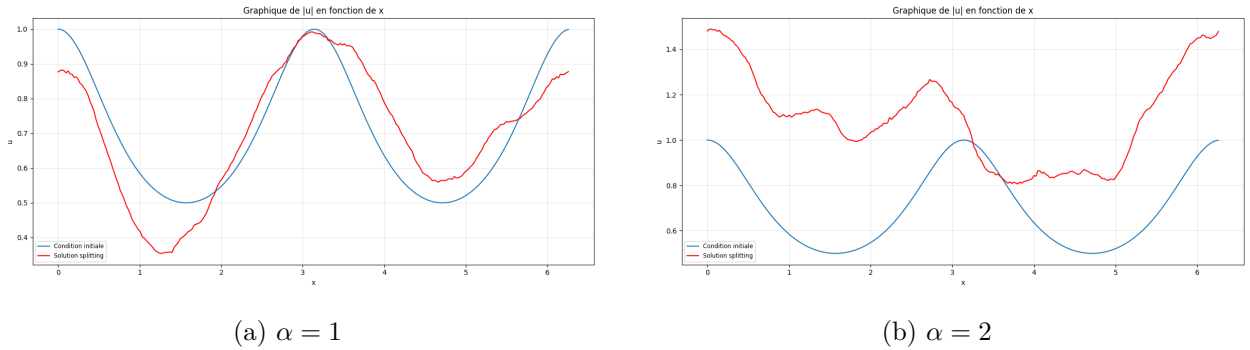


FIGURE 3.9 – $|u|$ schéma de splitting (interaction non locale)

Implémentons également la solution obtenue par la formule exacte 1.7, le schéma exponentiel stochastique 1.9, et le schéma de splitting pour l'équation de Schrödinger non linéaire déterministe. Les figures 3.10a et 3.10b permettent de comparer les résultats obtenus par les différentes méthodes. On peut remarquer que pour cette non linéarité le schéma exponentiel stochastique donne une solution presque exacte à celle du schéma de splitting.

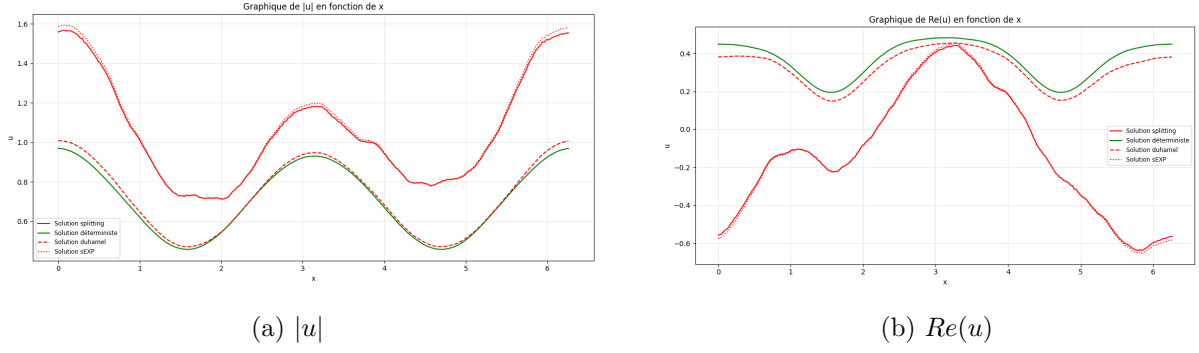


FIGURE 3.10 – Solution par différents schémas (interaction non locale)

Intéressons nous maintenant à l'évolution de l'espérance de la masse pour le schéma de splitting et le schéma exponentiel stochastique. L'algorithme de Monte Carlo pour le calcul des espérances est ici performé avec 7500 itérations. Comme voulu le schéma de splitting satisfait presque parfaitement la formule de la trace 2.5. De plus, pour la non linéarité interaction non locale cubique, le schéma exponentiel stochastique donne une évolution de la masse très proche de celle théorique. Ces deux observations peuvent être observées sur la figure 3.11. En calculant la norme infinie on a :

$$\|E(M(u_{n_{sp}})) - E(M(u_{n_{th}}))\|_{\infty} = 1.27e^{-2}, \quad (3.7)$$

ce qui est une erreur plus faible que celle obtenue pour la potentiel externe. Ces résultats sont cohérents avec ceux de [1] qui observent eux aussi des résultats presque parfaitement égaux de l'espérance théorique pour les deux schémas. Quand le temps augmente on peut cependant observer un léger écart se former pour le schéma exponentiel stochastique. Cela peut s'expliquer par des approximations moins précises, et notamment l'usage des transformées de Fourier pour la convolution de la non linéarité.

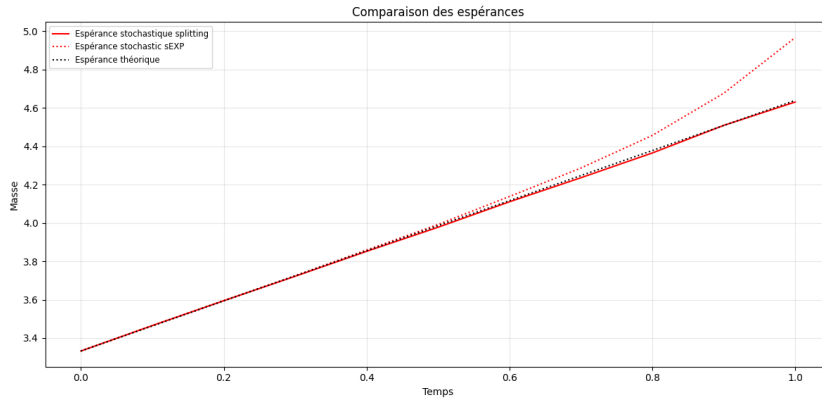


FIGURE 3.11 – Espérance de la masse obtenue par différents schémas

Enfin, précisons que si $u_0(x) = \frac{2}{2-\cos(x)}$ comme pour le potentiel externe, le schéma de splitting arrive à obtenir une solution, mais le schéma exponentiel ne converge pas du tout pour l'interaction non locale cubique.

Conclusion

Ce projet à permis d'étudier numériquement l'équation de Schrödinger non linéaire stochastique, à l'aide d'un schéma de type splitting, associé à une méthode pseudo-spectrale.

Les simulations réalisées ont confirmé les résultats obtenus par l'article de référence [1]. Le schéma d'Euler-Maruyama, intéressant car simple à mettre en place, n'a pas permis d'obtenir des résultats concluants. Cependant, il a été montré que le schéma de splitting respecte presque parfaitement la formule de la trace pour la masse, pour les deux non linéarité testées. De plus, on a pu comparer le schéma de splitting avec le schéma exponentiel stochastique. Ce deuxième schéma semble moins performant, et satisfait moins exactement la formule théorique pour l'évolution de la masse.

Il aurait été intéressant d'étudier plus théoriquement la convergence du schéma de splitting. Il serait aussi intéressant d'implémenter d'autres types de non linéarités, et d'autres types de bruits, par exemple multiplicatifs.

Ce projet m'a permis de mettre en pratique mes connaissances sur les équations différentielles partielles, tout en développant mes connaissances sur leurs versions stochastiques. J'ai également pu mettre en application mes connaissances sur l'implémentation de schémas numériques. J'ai apprécié travailler avec l'équation de Schrödinger, et découvrir plus en profondeur les schémas de type splitting.

Annexe A

Programmes

Les programmes peuvent être trouvés au lien suivant : Télécharger le fichier zip.
Ils sont également affichés ci-dessous.

A.1 Fonctions générales

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def u0(x): # external potential
5     u0 = 2.0/(2.0 - np.cos(x))
6     return u0
7
8 def V(x): # external potential
9     V = 3.0/(5.0-4.0*np.cos(x))
10    return V
11
12 def u0(x): # nonlocal interaction
13     u0 = 1/(1+(np.sin(x))**2)
14     return u0
15
16 def V(x): # nonlocal interaction
17     V = np.cos(x)
18     return V
19
20 def beta(nt, nx, tau):
21     dB = np.random.normal(0, np.sqrt(tau), size=(nt, nx))
22     dB[0, :] = 0.0
23     return dB
24
25 def gamma(k):
26     return 1.0/(1.0 + (k**2))
27
28 def e(k, x):
29     # k shape(K,), x shape(nx,) -> returns(K, nx)
30     return (1.0/np.sqrt(2.0*np.pi)) * np.exp(1j * np.outer(k, x))
31
32 def Wq(nt, tau, nx, x):
33     k = np.arange(nx)
34     beta_k = beta(nt, nx, tau) # shape(nt, nx)
35     g = gamma(k) # shape(nx,)
36     E = e(k, x) # shape(nx, nx)
37     W = np.sum(beta_k[:, :, None] * g[None, :, None] * E[None, :, :], axis=1)
38     return W
39
40 def laplacien(nx):
41     l=np.diagflat([2 for i in range(nx)]+np.diagflat([-1 for i in range(nx-1)],1)+np
42     .diagflat([-1 for i in range(nx-1)],-1)
43     l=1/(2*np.pi/nx)**2
44     return l
```



```

44
45 def noyau_chaleur(dt, nx):
46     k = np.fft.fftfreq(nx, d=(2.0*np.pi)/nx) * 2.0*np.pi
47     return np.exp(-1j * (k**2) * dt)
48
49 def init(f, n1, n2 = None, ite = None):
50     if n2 == None :
51         A = np.zeros(n1+2, dtype = complex)
52         A[1:n1+1]=f
53         A[0] = A[n1] # pour condition periodique : ajout fictive de la derniere
valeur avant la premiere
54         A[n1+1] = A[1]
55     else :
56         A = np.zeros((n1,n2+2), dtype = complex)
57         if len(np.shape(f)) == 1 :
58             for i in range(ite):
59                 A[i,1:n2+1]=f
60                 A[i,0] = A[i,n2]
61                 A[i,n2+1] = A[i,1]
62         elif len(np.shape(f)) == 2 :
63             for i in range(ite):
64                 A[i,1:n2+1]=f[i]
65                 A[i,0] = A[i,n2]
66                 A[i,n2+1] = A[i,1]
67         else :
68             print("f est de taille superieur a 2")
69     return A

```

A.2 Euler-Maruyama

```

1 alpha = 1
2 nx = 2**8
3 tau = 0.1
4 T = 1
5 nt = int(np.floor(T/tau)) + 1
6 x = np.linspace(0, 2*np.pi, nx, endpoint=False)
7 t = np.linspace(0, tau*(nt-1), nt)
8 dx = (2.0*np.pi)/nx
9
10 Vx = init(V(x), nx)
11
12 nc = 500 # nombre de realisations pour monte carlo
13 mass = np.zeros((nc,nt))
14
15 # Boucle Monte Carlo pour les calculs d'esperance
16 for i in range(nc):
17     # Initialisation
18     u = init(u0(x), nt, nx, 1)
19
20     W = init(Wq(nt, tau, nx, x), nt, nx, nt) # shape(nt, nx)
21
22     for n in range(1,nt):
23         # Schema de Euler-Maruyama
24         u_prev = u[n-1, :]
25
26         u[n, :] = u_prev - 1j*np.dot(laplacien(nx+2),u_prev) - 1j*tau*np.dot(Vx,
u_prev) - 1j*alpha*W[n]
27
28         u[n, 0] = u[n, nx] # pour periodicite
29         u[n, nx+1] = u[n, 1]
30
31     mass[i] = dx*np.sum(np.abs(u[:, 1:nx+1])**2, axis=1)
32
33 E_mass = 1/nc * np.sum(mass, axis=0)
34

```

```

35 #E(M) theorique
36 k = np.arange(nx)
37 TrQ = np.sum(gamma(k)**2)
38 E_mass_theorique = E_mass[0] + t*(alpha**2)*TrQ
39
40 plt.subplot(221)
41 plt.plot(x, np.real(u[0, 1:nx+1]), label="Temps initial", linewidth=1.5)
42 plt.plot(x, np.real(u[-1, 1:nx+1]), label="Temps final", linewidth=1.5)
43 plt.legend(fontsize="small")
44 plt.title("Graphique de Re(u) en fonction de x")
45 plt.xlabel("x")
46 plt.ylabel("u")
47 plt.grid(True, alpha=0.3)
48
49 plt.subplot(222)
50 plt.plot(x, np.abs(u[0, 1:nx+1]), label="Temps initial", linewidth=1.5)
51 plt.plot(x, np.abs(u[-1, 1:nx+1]), label="Temps final", linewidth=1.5)
52 plt.legend(fontsize="small")
53 plt.title("Graphique de |u| en fonction de x")
54 plt.xlabel("x")
55 plt.ylabel("u")
56 plt.grid(True, alpha=0.3)
57
58 plt.subplot(223)
59 plt.plot(x, np.abs(u[0, 1:nx+1]), label="Temps 0", linewidth=1.5)
60 plt.plot(x, np.abs(u[int(nt/2), 1:nx+1]), label="Temps "+str(int(nt/2)*tau),
        linewidth=1.5)
61 plt.plot(x, np.abs(u[-1, 1:nx+1]), label="Temps "+str(T), linewidth=1.5)
62 plt.legend(fontsize="small")
63 plt.title("Graphique de |u| au fil de t")
64 plt.xlabel("x")
65 plt.ylabel("u")
66 plt.grid(True, alpha=0.3)
67
68 plt.subplot(224)
69 plt.plot(t, E_mass, "+r", label="Esperance Euler-Maruyama", linewidth=1.5)
70 plt.plot(t, E_mass_theorique, ":k", label="Esperance theorique", linewidth=1.5)
71 plt.legend(fontsize="small")
72 plt.title("Comparaison des esperances")
73 plt.xlabel("Temps")
74 plt.ylabel("Masse")
75 plt.grid(True, alpha=0.3)
76
77 plt.suptitle("Schema d'Euler Maruyama, tau = "+str(tau)+", nx = "+str(nx))
78 plt.tight_layout()
79 plt.show()

```

A.3 Potentiel externe

```

1 alpha = 1.0
2 nx = 2**8
3 tau = 0.1
4 T = 1
5 nt = int(np.floor(T/tau)) + 1
6 x = np.linspace(0, 2*np.pi, nx, endpoint=False)
7 t = np.linspace(0, tau*(nt-1), nt)
8 dx = (2.0*np.pi)/nx
9
10 Vx = init(V(x), nx)
11 S = init(noyau_chaleur(tau,nx), nx)
12
13 nc = 500 # nombre de realisations pour monte carlo
14 mass_splitting = np.zeros((nc, nt))
15 mass_duhamel = np.zeros((nc, nt))
16 mass_sEXP = np.zeros((nc, nt))

```

```

17
18 # Boucle Monte Carlo pour les calculs d'esperance
19 for i in range(nc):
20     # Initialisation
21     u = init(u0(x), nt, nx, 1)
22     u_duhamel = init(u0(x), nt, nx, 1)
23     u_sEXP = init(u0(x), nt, nx, 1)
24
25     W = init(Wq(nt, tau, nx, x), nt, nx, nt)
26
27     for n in range(1, nt):
28         # Schema de splitting
29         u_prev = u[n-1, :]
30         u_bis = np.exp(-1j*tau*Vx)*u_prev
31         u_bis = u_bis - 1j*alpha*W[n]
32         u_lin = np.fft.ifft(S*np.fft.fft(u_bis))
33         u[n, :] = u_lin
34         u[n, 0] = u[n, nx]
35         u[n, nx+1] = u[n, 1]
36
37         # Formule de Duhamel
38         temps = t[n]
39         integrale1 = np.zeros(nx+2, dtype=complex)
40         integrale2 = np.zeros(nx+2, dtype=complex)
41         for l in range(1, n+1):
42             s = tau*(l-1)
43             Ss = init(noyau_chaleur(temps-s, nx), nx)
44             integrale1 += np.fft.ifft(Ss*np.fft.fft(Vx*u_duhamel[l-1,:])) * tau
45             integrale2 += np.fft.ifft(Ss*np.fft.fft(W[l])) * tau
46             St = init(noyau_chaleur(temps, nx), nx)
47             u_init = init(u0(x), nx)
48             u_duhamel[n, :] = np.fft.ifft(St*np.fft.fft(u_init)) - 1j*integrale1 - 1j*
alpha*integrale2
49             u_duhamel[n, 0] = u_duhamel[n, nx]
50             u_duhamel[n, nx+1] = u_duhamel[n, 1]
51
52         # Schema stochastique exponentiel
53         u_sEXP_prev = u_sEXP[n-1, :]
54         u_sEXP_bis = u_sEXP_prev - 1j*tau*Vx*u_sEXP_prev - 1j*alpha*W[n]
55         u_sEXP_lin = np.fft.ifft(S*np.fft.fft(u_sEXP_bis))
56         u_sEXP[n, :] = u_sEXP_lin
57         u_sEXP[n, 0] = u_sEXP[n, nx]
58         u_sEXP[n, nx+1] = u_sEXP[n, 1]
59
60         mass_splitting[i] = dx*np.sum(np.abs(u[:, 1:nx+1])**2, axis=1)
61         mass_duhamel[i] = dx*np.sum(np.abs(u_duhamel[:, 1:nx+1])**2, axis=1)
62         mass_sEXP[i] = dx*np.sum(np.abs(u_sEXP[:, 1:nx+1])**2, axis=1)
63
64     E_mass_splitting = 1/nc * np.sum(mass_splitting, axis=0)
65     E_mass_duhamel = 1/nc * np.sum(mass_duhamel, axis=0)
66     E_mass_sEXP = 1/nc * np.sum(mass_sEXP, axis=0)
67
68     k = np.arange(nx)
69     TrQ = np.sum(gamma(k)**2)
70     E_mass_theorique = E_mass_splitting[0] + t*(alpha**2)*TrQ
71
72 # Implementation du schema deterministe (alpha = 0)
73 u_det = init(u0(x), nt, nx, 1)
74
75 for m in range(1, nt):
76     # Schema deterministe
77     u_prev_det = u_det[m-1, :]
78     u_bis_det = np.exp(-1j*tau*Vx)*u_prev_det
79     u_lin_det = np.fft.ifft(S * np.fft.fft(u_bis_det))
80
81     u_det[m, :] = u_lin_det
82     u_det[m, 0] = u_det[m, nx]

```

```

83     u_det[m, nx+1] = u_det[m, 1]
84
85 mass_det = dx * np.sum(np.abs(u_det[:, 1:nx+1])**2, axis=1)
86
87 erreur_L2_final = np.linalg.norm(u[-1, 1:nx+1] - u_duhamel[-1, 1:nx+1], ord=2)
88 print("Erreur schema splitting et formule duhamel (t=1) :", erreur_L2_final)
89 print("Difference mass :", np.linalg.norm(E_mass_splitting - E_mass_theorique, ord=np
    .inf))
90
91 plt.subplot(221)
92 plt.plot(x, np.real(u[0,1:nx+1]), label="Condition initiale", linewidth=1.5)
93 plt.plot(x, np.real(u[-1, 1:nx+1]), "r", label="Solution splitting", linewidth=1.5)
94 plt.plot(x, np.real(u_det[-1, 1:nx+1]), "g", label="Solution deterministe", linewidth
    =1.5)
95 plt.plot(x, np.real(u_duhamel[-1, 1:nx+1]), "--r", label="Solution duhamel",
    linewidth=1.5)
96 plt.plot(x, np.real(u_sEXP[-1, 1:nx+1]), ":r", label="Solution sEXP", linewidth=1.5)
97 plt.legend(fontsize="small")
98 plt.title("Graphique de Re(u) en fonction de x")
99 plt.xlabel("x")
100 plt.ylabel("u")
101 plt.grid(True, alpha=0.3)
102
103 plt.subplot(222)
104 plt.plot(x, np.abs(u[0,1:nx+1]), label="Condition initiale", linewidth=1.5)
105 plt.plot(x, np.abs(u[-1, 1:nx+1]), "r", label="Solution splitting", linewidth=1.5)
106 plt.plot(x, np.abs(u_det[-1, 1:nx+1]), "g", label="Solution deterministe", linewidth
    =1.5)
107 plt.plot(x, np.abs(u_duhamel[-1, 1:nx+1]), "--r", label="Solution duhamel", linewidth
    =1.5)
108 plt.plot(x, np.abs(u_sEXP[-1, 1:nx+1]), ":r", label="Solution sEXP", linewidth=1.5)
109 plt.legend(fontsize="small")
110 plt.title("Graphique de |u| en fonction de x")
111 plt.xlabel("x")
112 plt.ylabel("u")
113 plt.grid(True, alpha=0.3)
114
115 plt.subplot(223)
116 plt.plot(x, np.abs(u[0, 1:nx+1]), label="Temps 0", linewidth=1.5)
117 plt.plot(x, np.abs(u[int(nt/3), 1:nx+1]), label="Temps "+str(round(int(nt/3)*tau,2)),
    linewidth=1.5)
118 plt.plot(x, np.abs(u[int(2*nt/3), 1:nx+1]), label="Temps "+str(round(int(2*nt/3)*tau
    ,2)), linewidth=1.5)
119 plt.plot(x, np.abs(u[-1, 1:nx+1]), label="Temps "+str(T), linewidth=1.5)
120 plt.legend(fontsize="small")
121 plt.title("Graphique de |u| splitting")
122 plt.xlabel("x")
123 plt.ylabel("u")
124 plt.grid(True, alpha=0.3)
125
126 plt.subplot(224)
127 plt.plot(t, mass_det, "g", label="Cas deterministe (alpha=0)", linewidth=1.5)
128 plt.plot(t, E_mass_splitting, "red", label="Esperance stochastique splitting",
    linewidth=1.5)
129 plt.plot(t, E_mass_duhamel, "--r", label="Esperance stochastique duhamel", linewidth
    =1.5)
130 plt.plot(t, E_mass_sEXP, ":r", label="Esperance stochastique sEXP", linewidth=1.5)
131 plt.plot(t, E_mass_theorique, ":k", label="Esperance theorique", linewidth=1.5)
132 plt.legend(fontsize="small")
133 plt.title("Comparaison des esperances")
134 plt.xlabel("Temps")
135 plt.ylabel("Masse")
136 plt.grid(True, alpha=0.3)
137
138 plt.suptitle("tau = "+str(tau)+" , nx = "+str(nx))
139 plt.tight_layout()
140 plt.show()

```

A.4 Interaction non locale

```
1 alpha = 1.0
2 nx = 2**8
3 tau = 0.1
4 T = 1
5 nt = int(np.floor(T/tau)) + 1
6 x = np.linspace(0, 2*np.pi, nx, endpoint=False)
7 t = np.linspace(0, tau*(nt-1), nt)
8 dx = (2.0*np.pi)/nx
9
10 S = init(noyau_chaleur(tau, nx), nx)
11
12 nc = 500 # nombre de realisations pour monte carlo
13 mass_splitting = np.zeros((nc, nt))
14 mass_duhamel = np.zeros((nc, nt))
15 mass_sEXP = np.zeros((nc, nt))
16
17 # Boucle Monte Carlo pour les calculs d'esperance
18 for i in range(nc):
19     # Initialisation
20     u = init(u0(x), nt, nx, 1)
21     u_duhamel = init(u0(x), nt, nx, 1)
22     u_sEXP = init(u0(x), nt, nx, 1)
23
24     W = init(Wq(nt, tau, nx, x), nt, nx, nt) # shape(nt, nx)
25
26     for n in range(1, nt):
27         # Schema de splitting
28         u_prev = u[n-1, :]
29         Vx_hat = np.fft.fft(np.abs(u_prev)**2)*np.fft.fft(init(V(x), nx))
30         Vx = np.fft.ifft(Vx_hat)*dx
31         u_bis = np.exp(-1j*tau*Vx)*u_prev
32         u_bis = u_bis - 1j*alpha*W[n]
33         u_lin = np.fft.ifft(S*np.fft.fft(u_bis))
34         u[n, :] = u_lin
35         u[n, 0] = u[n, nx]
36         u[n, nx+1] = u[n, 1]
37
38         # Formule de Duhamel
39         temps = t[n]
40         integrale1 = np.zeros(nx+2, dtype = complex)
41         integrale2 = np.zeros(nx+2, dtype = complex)
42         for l in range(1, n+1):
43             s = tau*(l-1)
44             Ss = init(noyau_chaleur(temps-s, nx), nx)
45             Vx_hat_duhamel = np.fft.fft(np.abs(u_duhamel[l-1, :])**2)*np.fft.fft(init(
V(x), nx))
46             Vx_duhamel = np.fft.ifft(Vx_hat_duhamel) * dx
47             integrale1 += np.fft.ifft(Ss*np.fft.fft(Vx_duhamel*u_duhamel[l-1, :])) *
tau
48             integrale2 += np.fft.ifft(Ss*np.fft.fft(W[l])) * tau
49             St = init(noyau_chaleur(temps, nx), nx)
50             u_init = init(u0(x), nx)
51             u_duhamel[n, :] = np.fft.ifft(St*np.fft.fft(u_init)) - 1j*integrale1 - 1j*
alpha*integrale2
52             u_duhamel[n, 0] = u_duhamel[n, nx]
53             u_duhamel[n, nx+1] = u_duhamel[n, 1]
54
55         # Schema stochastique exponentiel
56         u_sEXP_prev = u_sEXP[n-1, :]
57         Vx_hat_sEXP = np.fft.fft(np.abs(u_sEXP_prev)**2)*np.fft.fft(init(V(x), nx))
58         Vx_sEXP = np.fft.ifft(Vx_hat_sEXP)*dx
59         u_sEXP_bis = u_sEXP_prev - 1j*tau*Vx_sEXP*u_sEXP_prev - 1j*alpha*W[n]
60         u_sEXP_lin = np.fft.ifft(S*np.fft.fft(u_sEXP_bis))
61         u_sEXP[n, :] = u_sEXP_lin
62         u_sEXP[n, 0] = u_sEXP[n, nx]
```

```

63     u_sEXP[n, nx+1] = u_sEXP[n, 1]
64
65     mass_splitting[i] = dx*np.sum(np.abs(u[:, 1:nx+1])**2, axis=1)
66     mass_duhamel[i] = dx*np.sum(np.abs(u_duhamel[:, 1:nx+1])**2, axis=1)
67     mass_sEXP[i] = dx*np.sum(np.abs(u_sEXP[:, 1:nx+1])**2, axis=1)
68
69 E_mass_splitting = 1/nc * np.sum(mass_splitting, axis=0)
70 E_mass_duhamel = 1/nc * np.sum(mass_duhamel, axis=0)
71 E_mass_sEXP = 1/nc * np.sum(mass_sEXP, axis=0)
72
73 k = np.arange(nx)
74 TrQ = np.sum(gamma(k)**2)
75 E_mass_theorique = E_mass_splitting[0] + t*(alpha**2)*TrQ
76
77 # Implementation du schema deterministe (alpha = 0)
78 u_det = init(u0(x), nt, nx, 1)
79
80 for m in range(1, nt):
81     # Schema deterministe
82     u_prev_det = u_det[m-1, :]
83     Vx_hat_det = np.fft.fft(np.abs(u_prev_det)**2)*np.fft.fft(init(V(x), nx))
84     Vx_det = np.fft.ifft(Vx_hat_det)*dx
85     u_bis_det = np.exp(-1j*tau*Vx_det)*u_prev_det
86     u_lin_det = np.fft.ifft(S * np.fft.fft(u_bis_det))
87
88     u_det[m, :] = u_lin_det
89     u_det[m, 0] = u_det[m, nx]
90     u_det[m, nx+1] = u_det[m, 1]
91
92 mass_det = dx * np.sum(np.abs(u_det[:, 1:nx+1])**2, axis=1)
93
94 erreur_L2_final = np.linalg.norm(u[-1, 1:nx+1] - u_duhamel[-1, 1:nx+1], ord=2)
95 print("Erreur schema splitting et formule duhamel (t=1) :", erreur_L2_final)
96 print("Difference mass :", np.linalg.norm(E_mass_splitting - E_mass_theorique, ord=np
    .inf))
97
98 plt.subplot(221)
99 plt.plot(x, np.real(u[0, 1:nx+1]), label="Condition initiale", linewidth=1.5)
100 plt.plot(x, np.real(u[-1, 1:nx+1]), "r", label="Solution splitting", linewidth=1.5)
101 plt.plot(x, np.real(u_det[-1, 1:nx+1]), "g", label="Solution deterministe", linewidth
    =1.5)
102 plt.plot(x, np.real(u_duhamel[-1, 1:nx+1]), "--r", label="Solution duhamel",
    linewidth=1.5)
103 plt.plot(x, np.real(u_sEXP[-1, 1:nx+1]), ":r", label="Solution sEXP", linewidth=1.5)
104 plt.legend(fontsize="small")
105 plt.title("Graphique de Re(u) en fonction de x")
106 plt.xlabel("x")
107 plt.ylabel("u")
108 plt.grid(True, alpha=0.3)
109
110 plt.subplot(222)
111 plt.plot(x, np.abs(u[0, 1:nx+1]), label="Condition initiale", linewidth=1.5)
112 plt.plot(x, np.abs(u[-1, 1:nx+1]), "r", label="Solution splitting", linewidth=1.5)
113 plt.plot(x, np.abs(u_det[-1, 1:nx+1]), "g", label="Solution deterministe", linewidth
    =1.5)
114 plt.plot(x, np.abs(u_duhamel[-1, 1:nx+1]), "--r", label="Solution duhamel", linewidth
    =1.5)
115 plt.plot(x, np.abs(u_sEXP[-1, 1:nx+1]), ":r", label="Solution sEXP", linewidth=1.5)
116 plt.legend(fontsize="small")
117 plt.title("Graphique de |u| en fonction de x")
118 plt.xlabel("x")
119 plt.ylabel("u")
120 plt.grid(True, alpha=0.3)
121
122 plt.subplot(223)
123 plt.plot(x, np.abs(u[0, 1:nx+1]), label="Temps 0", linewidth=1.5)
124 plt.plot(x, np.abs(u[int(nt/3), 1:nx+1]), label="Temps "+str(round(int(nt/3)*tau, 2)),

```

```

        linewidth=1.5)
125 plt.plot(x, np.abs(u[int(2*nt/3), 1:nx+1]), label="Temps "+str(round(int(2*nt/3)*tau
,2)), linewidth=1.5)
126 plt.plot(x, np.abs(u[-1, 1:nx+1]), label="Temps "+str(T), linewidth=1.5)
127 plt.legend(fontsize="small")
128 plt.title("Graphique de |u| splitting")
129 plt.xlabel("x")
130 plt.ylabel("u")
131 plt.grid(True, alpha=0.3)
132
133 plt.subplot(224)
134 plt.plot(t, mass_det, "g", label="Cas deterministe (alpha=0)", linewidth=1.5)
135 plt.plot(t, E_mass_splitting, "red", label="Esperance stochastique splitting",
linewidth=1.5)
136 plt.plot(t, E_mass_duhamel, "--r", label="Esperance stochastique duhamel", linewidth
=1.5)
137 plt.plot(t, E_mass_sEXP, ":r", label = "Esperance stochastique sEXP", linewidth=1.5)
138 plt.plot(t, E_mass_theorique, ":k", label="Esperance theorique", linewidth=1.5)
139 plt.legend(fontsize="small")
140 plt.title("Comparaison des esperances")
141 plt.xlabel("Temps")
142 plt.ylabel("Masse")
143 plt.grid(True, alpha=0.3)
144
145 plt.suptitle("tau = "+str(tau)+" , nx = "+str(nx))
146 plt.tight_layout()
147 plt.show()

```

Bibliographie

- [1] C.-E. Bréhier and D. Cohen, “Analysis of a splitting scheme for a class of nonlinear stochastic schrödinger equations,” *Applied Numerical Mathematics*, vol. 186, pp. 57–83, Apr. 2023. [Online]. Available : <https://linkinghub.elsevier.com/retrieve/pii/S0168927423000028>
- [2] R. Illner, P. F. Zweifel, and H. Lange, “Global existence, uniqueness and asymptotic behaviour of solutions of the wigner-poisson and schrödinger-poisson systems,” *Mathematical Methods in the Applied Sciences*, vol. 17, no. 5, pp. 349–376, Apr. 1994. [Online]. Available : <https://onlinelibrary.wiley.com/doi/10.1002/mma.1670170504>
- [3] C. Negulescu, “L’équation de schrödinger.” [Online]. Available : <https://www.math.univ-toulouse.fr/~cnegules/Article/CoursSchroedinger.pdf>
- [4] V. V. Konotop and L. Vazquez, *Nonlinear Random Waves*. World Scientific, 1994.
- [5] A. de Bouard and A. Debussche, “The stochastic nonlinear schrödinger equation in h^1 ,” *Stochastic Analysis and Applications*, vol. 21, no. 1, pp. 97–126, Jan. 2003. [Online]. Available : <https://doi.org/10.1081/SAP-120017534>
- [6] A. de Bouard, “Theoretical and numerical aspects of stochastic nonlinear schrödinger equations,” *Monte Carlo Methode and Appl.*, vol. 7, no. 1-2, pp. 55–63, 2001.
- [7] A. Debussche and L. Di Menza, “Numerical simulation of focusing stochastic nonlinear schrödinger equations,” *Physica D : Nonlinear Phenomena*, vol. 162, no. 3-4, pp. 131–154, Feb. 2002. [Online]. Available : <https://linkinghub.elsevier.com/retrieve/pii/S0167278901003797>
- [8] A. De Bouard and A. Debussche, “Weak and strong order of convergence of a semidiscrete scheme for the stochastic nonlinear schrodinger equation,” *Applied Mathematics and Optimization*, vol. 54, no. 3, pp. 369–399, Nov. 2006. [Online]. Available : <http://link.springer.com/10.1007/s00245-006-0875-0>
- [9] J. Li and X. Li, “Exponential integrators for stochastic schrödinger equation,” *Physical Review E*, vol. 101, no. 1, p. 013312, Jan. 2020. [Online]. Available : <http://arxiv.org/abs/1911.07374>
- [10] R. Anton and D. Cohen, “Exponential integrators for stochastic schrödinger equations driven by ito noise,” Jan. 2016. [Online]. Available : <http://arxiv.org/abs/1601.06623>
- [11] R. Marty, “On a splitting scheme for the nonlinear schrödinger equation in a random medium,” *Communications in Mathematical Sciences*, vol. 4, no. 4, pp. 679–705, 2006. [Online]. Available : <https://link.intlpress.com/JDetail/1806266221173624834>
- [12] C.-E. Bréhier and D. Cohen, “Strong rates of convergence of a splitting scheme for schrödinger equations with nonlocal interaction cubic nonlinearity and white noise dispersion,” *SIAM/ASA Journal on Uncertainty Quantification*, vol. 10, no. 1, pp. 453–480, Mar. 2022. [Online]. Available : <https://epubs.siam.org/doi/10.1137/20M1378168>
- [13] C. Romero-Melendez and D. Castillo-Fernandez, “A stochastic controlled schrödinger equation : convergence and robust stability for numerical solutions,” *Cybernetics and Physics*, vol. 10, no. 3, pp. 178–184, 2021. [Online]. Available : <https://scholar.google.com/scholar?oi=bibs&cluster=6468292450727470094&btnI=1&hl=es>